

Uniwersytet Gdański
Wydział Matematyki, Fizyki i Informatyki

Dawid Kacper Odolczyk
nr albumu: 298988
kierunek studiów: informatyka

**Analiza porównawcza
i integracyjna mechanizmów
Project Loom z istniejącymi
modelami współbieżności
w środowisku JVM w kontekście
wybranych scenariuszy zastosowań**

Praca magisterska
wykonana pod kierunkiem
dra Tomasza Borzyszkowskiego

Gdańsk 2026

Spis treści

Streszczenie	5
1 Wstęp	7
1.1 Motywacja	7
1.2 Cel i zakres pracy	7
1.3 Pytania badawcze	7
1.4 Struktura pracy	7
2 Wprowadzenie teoretyczne	9
2.1 Podstawowe pojęcia	9
2.1.1 Współbieżność a równoległość	9
2.1.2 Pojęcie i zasada działania wątków	10
2.2 Problemy występujące w systemach współbieżnych	10
2.2.1 Wzajemne wykluczanie i synchronizacja	11
2.2.2 <i>Race condition</i> i pamięć współdzielona	11
2.2.3 Problemy żywotności: <i>deadlock</i> , <i>livelock</i> , <i>starvation</i>	11
2.2.4 Problemy związane z wydajnością	11
2.3 Modele współbieżności w środowisku JVM	11
2.3.1 Klasyczny model współbieżności (Thread API)	12
2.3.2 Model oparty o pule wątków (<code>ExecutorService</code>)	12
2.3.3 Reaktywny model współbieżności (Reactive Streams, Project Reactor, RxJava)	12
3 Mechanizmy współbieżności oferowane przez Project Loom	13
3.1 Wprowadzenie do Project Loom	13
3.2 Wirtualne wątki (<i>Virtual Threads</i>)	14
3.3 Współbieżność strukturalna (<i>Structured Concurrency</i>)	16
3.4 Zmienne zakresowe (<i>Scoped Values</i>)	16
4 Stan badań i analiza literatury	17
4.1 Dotychczasowe badania nad Project Loom	17
4.2 Charakterystyka i wyniki dotychczasowych badań	17

4.2.1	Obsługa dużej liczby równoczesnych żądań HTTP	17
4.2.2	Równoległe przetwarzanie złożonych obliczeń	17
4.3	Zdefiniowanie luki badawczej i kierunku własnych badań	17
5	Metodyka badań	19
5.1	Cel i zakres badań	19
5.2	Szczegółowy opis scenariuszy testowych	19
5.2.1	Scenariusz A: Integracja z bazą danych ze sterownikiem blokującym (JDBC) i reaktywnym (R2DBC)	19
5.2.2	Scenariusz B: Agregacja z użyciem mechanizmu kworum (QS) i propagacją kontekstu	19
6	Wyniki badań i analiza	21
6.1	Metody pomiaru i kryteria oceny	21
6.2	Wyniki pomiarów dla poszczególnych scenariuszy	21
6.3	Analiza porównawcza i interpretacja wyników	21
7	Wnioski i rekomendacje	29
7.1	Odpowiedzi na pytania badawcze	29
7.2	Rekomendacje i wzorce zastosowań mechanizmów Project Loom . . .	29
7.3	Ograniczenia badań i propozycje dalszych badań	29
8	Podsumowanie	31
	Bibliografia	33
	Spis rysunków	34
	Spis tabel	35
	Spis kodów źródłowych	36

Streszczenie

Celem niniejszej pracy jest ocena wpływu mechanizmów wirtualnych wątków i współbieżności strukturalnej, będących częścią inicjatywy Project Loom, na wydajność oraz złożoność implementacyjną aplikacji współbieżnych w środowisku JVM. W pracy analizowane są trzy główne podejścia do współbieżności: wirtualne wątki (Project Loom), Reactive Streams oraz Kotlin Coroutines. Badanie opiera się na realistycznych scenariuszach produkcyjnych, obejmujących obsługę równoczesnych żądań HTTP, integrację z zewnętrznymi API, przetwarzanie danych w bazach danych oraz intensywne przetwarzanie strumieniowe danych telemetrycznych.

Praca koncentruje się zarówno na aspektach technicznych (przepustowość, opóźnienia, zużycie zasobów), jak i implementacyjnych (czytelność kodu, złożoność architektury, propagacja kontekstu i błędów). Wyniki testów pozwalają na ocenę, w jakim stopniu wirtualne wątki upraszczają projektowanie aplikacji współbieżnych, a także na sformułowanie praktycznych rekomendacji dla zespołów developerskich rozważających migrację do Loom lub hybrydowe podejścia współbieżności w JVM.

Abstract

“A Comparative and Integrative Analysis of Project Loom Mechanisms and Existing Concurrency Models in the JVM Environment for Selected Application Scenarios”

The engineering thesis describes the development of a mobile application made using the Flutter framework. The application is designed for scanning barcodes of food products to obtain detailed information about them. In addition to providing product information such as ingredients, allergens presence, and nutritional value, the application allows users to determine if a product is suitable for them based on their dietary restrictions and preferences. The application features a system for retrieving product information, which reduces the need to query external APIs. Furthermore, the application does not require an Internet connection to be functional.

The thesis begins by addressing the need for easier access to information about food products' ingredients, especially among individuals with food intolerances. Subsequently, the thesis provides a detailed description of the items present on food products' labels, such as the structure of EAN barcodes, types of allergens and nutritional values along with standards for their presentation, and a detailed description of the Nutri-Score system. Next, it describes the technologies used in developing the application and the data structure it relies on, including information about products available in the Open Food Facts database. The thesis further focuses on the application's architecture, programming practices used during its development, including widget-based architecture with detailed descriptions of their functionality, as well as the process of retrieving and storing product information. The thesis concludes with insights into the application's development process and suggestions for future improvements.

Rozdział 1

Wstęp

[1] bla bla bla [2]

Jak podają Veen i Vlijmincx [3], lorem ipsum cos tam bla bla bla.

```
Runnable task = () -> System.out.println("Hello, World!");  
Thread.startVirtualThread(task);
```

Listing 1: Uruchamianie wirtualnego wątku

(źródło: opracowanie własne na podstawie [3])

1.1 Motywacja

1.2 Cel i zakres pracy

1.3 Pytania badawcze

1.4 Struktura pracy

Rozdział 2

Wprowadzenie teoretyczne

Drugi rozdział pracy poświęcony jest teoretycznemu omówieniu podstawowych pojęć związanych z równoległością i współbieżnością, w tym w omówieniu działania wątków we współczesnych systemach operacyjnych, zdefiniowaniu typowych problemów występujących w systemach współbieżnych oraz przedstawieniu najważniejszych założeń różnych modeli współbieżności dostępnych w ramach środowiska JVM, w szczególności klasycznego modelu współbieżności oferowanego w ramach Thread API, modelu opartego o pulę wątków (dostępnego w ramach interfejsu `ExecutorService`) oraz reaktywnego modelu współbieżności oferowanego przez Reactive Streams i RxJava.

2.1 Podstawowe pojęcia

2.1.1 Współbieżność a równoległość

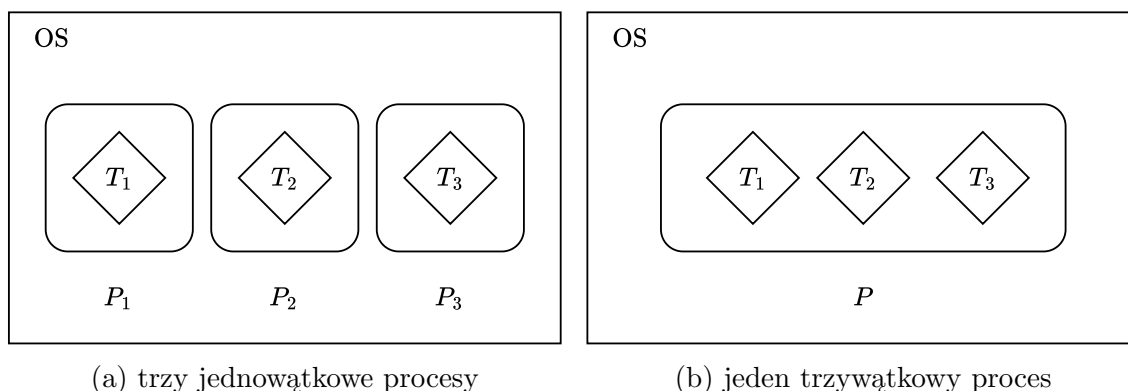
Jak podają Tanenbaum i Bos [4], *współbieżność* (ang. *concurrency*) oznacza zdolność systemu do obsługi wielu zadań w taki sposób, że ich wykonywanie nakłada się w czasie, nawet jeśli w danym momencie wykonywane jest tylko jedno z nich. *Równoległość* (ang. *parallelism*) oznacza obsługę wielu zadań w tym samym czasie przy wykorzystaniu wielu jednostek obliczeniowych.

Innymi słowy, współbieżność odnosi się do strukturalnej organizacji systemu oraz sposobu modelowania i zarządzania wieloma zadaniami, natomiast równoległość oznacza ich faktyczne, jednoczesne wykonywanie na wielu procesorach lub rdzeniach. We współczesnych systemach operacyjnych zadania te są wykonywane w ramach *procesów* i *wątków*.

2.1.2 Pojęcie i zasada działania wątków

We współczesnych systemach operacyjnych najważniejszą abstrakcją umożliwiającą współbieżne wykonywanie operacji są *procesy* (ang. *processes*). Jak wskazują Tanenbaum i Bos [4], procesy umożliwiają przekształcenie jednej jednostki obliczeniowej (CPU) na wiele wirtualnych jednostek, w ramach których mogą współbieżnie wykonywać zadania poprzez umiejętność CPU do szybkiego przełączania się między procesami.

W celu umożliwienia bardziej efektywnej współbieżności wewnątrz pojedynczego procesu, współczesne systemy operacyjne oferują *wątki* (ang. *threads*), definiowane jako lżejsze jednostki wykonania. Wątki działają w obrębie jednego procesu i współdzielą z nim przestrzeń adresową oraz zasoby, zachowując jednocześnie własny kontekst wykonania [4]. Różnice między procesem a wątkiem prezentuje rysunek 2.1.



(a) trzy jednowątkowe procesy

(b) jeden trzywątkowy proces

Rysunek 2.1: Rozróżnienie procesów i wątków w systemie operacyjnym

(źródło: opracowanie własne na podstawie [4])

Mimo że oba systemy przedstawione powyżej operują na trzech wątkach T_1 , T_2 i T_3 , wątki na rysunku 2.1a operują na odrębnych przestrzeniach adresowych, podczas gdy wątki przedstawione na rysunku 2.1b działają w obrębie jednej (wspólnej) przestrzeni adresowej, co wynika z faktu operowania wewnątrz jednego (wspólnego) procesu P .

2.2 Problemy występujące w systemach współbieżnych

Mimo wielu zalet wykorzystania wątków do współbieżnego wykonywania zadań, podejście to wiąże się z istnieniem wielu problemów obecnych w systemach współbieżnych. Wiele publikacji wskazuje na cztery główne kategorie problemów: problemy związane z mechanizmem wzajemnego wykluczania i synchronizacją

wątków, *race condition* i pamięć współdzieloną, problemy żywotności, takie jak *deadlock*, *livelock* i *starvation*, a także problemy związane z wydajnością. Poniżej opisano charakterystykę każdego z nich.

2.2.1 Wzajemne wykluczanie i synchronizacja

Ben-Ari [5] definiuje wzajemne wykluczanie (ang. *mutual exclusion*) jako jeden z najważniejszych problemów programowania współbieżnego z racji bycia abstrakcją dla wielu problemów związanych z synchronizacją wątków.

Herlihy i Shavit [6] wskazują, że wątki *wykluczają się wzajemnie*, jeśli co najmniej dwa z nich próbują równocześnie wykonać wspólny blok kodu, nazywany *sekcją krytyczną* (ang. *critical section*), ale w danym momencie tylko jeden z nich ma dostęp do sekcji krytycznej.

2.2.2 *Race condition* i pamięć współdzielona

2.2.3 Problemy żywotności: *deadlock*, *livelock*, *starvation*

Goetz [7] definiuje *żywotność* (ang. *liveness*) jako właściwość systemu, która zapewnia, że koniec końców pewna własność się wydarzy, czyli że system nie będzie pozostawał w stanie zawieszenia lub bezczynności. Problemy żywotności występują, gdy system nie jest w stanie kontynuować wykonywania zadań z powodu wzajemnych zależności między wątkami lub braku zasobów. Najpopularniejszymi z nich są *deadlock*, *livelock* i *starvation*.

2.2.4 Problemy związane z wydajnością

2.3 Modele współbieżności w środowisku JVM

Środowisko JVM oferuje wiele różnych modeli umożliwiających tworzenie współbieżnych aplikacji. Ta część poświęcona jest omówieniu najpopularniejszych podejść, w tym podstawowego (i chronologicznie pierwszego) modelu, dostępnego w ramach Thread API, modelu opartego o wykorzystanie wzorca projektowego *Pula* (ang. *Pool*), dostępnego w ramach interfejsu `ExecutorService`, a także modelu reaktywnego.

- 2.3.1 Klasyczny model współbieżności (Thread API)**
- 2.3.2 Model oparty o pulę wątków (ExecutorService)**
- 2.3.3 Reaktywny model współbieżności (Reactive Streams, Project Reactor, RxJava)**

Rozdział 3

Mechanizmy współbieżności oferowane przez Project Loom

Trzeci rozdział pracy poświęcony jest omówieniu najważniejszych mechanizmów dostępnych w ramach Project Loom – inicjatywy usprawniającej tworzenie aplikacji współbieżnych w środowisku JVM. Rozdział zawiera zwięzłą prezentację historii powstawania tego projektu, a także omówienie najważniejszych mechanizmów, które on oferuje, tj. odrębną kategorię wątków, określanych mianem wątków wirtualnych (ang. *virtual threads*), mechanizm współbieżności strukturalnej (ang. *structured concurrency*) i mechanizm przekazywania danych kontekstowych w ramach zmiennych zakresowych (ang. *scoped values*).

3.1 Wprowadzenie do Project Loom

Project Loom jest inicjatywą zapoczątkowaną pod koniec 2017 roku przez OpenJDK. W ramach projektu wyróżnia się trzy główne składowe:

- mechanizm *wirtualnych wątków* (ang. *virtual threads*), będący alternatywą dla klasycznego mechanizmu wątków (wątków platformowych) i zdefiniowany w dokumencie JEP¹ o numerze 444 [8],
- mechanizm *współbieżności strukturalnej* (ang. *structured concurrency*), umożliwiający uporządkowanie pracy z wątkami poprzez dodanie tak zwanych *zakresów zadań* (ang. *task scopes*) i strategii zachowań w przypadku wystąpienia błędu lub przerwania działania wątku, opisany w dokumencie JEP 525 ² [9],

¹JEP (*JDK Enhancement Proposal*) to formalny dokument opisujący proponowane zmiany, nowe funkcjonalności lub usprawnienia platformy JDK.

²W chwili publikacji najnowsza wersja mechanizmu współbieżności strukturalnej została udostępniona w wersji *preview*, co oznacza, że jest dostępna w standardowym JDK, lecz jej składnia lub semantyka mogą ulec zmianie przed ostatecznym włączeniem do specyfikacji platformy Java.

- mechanizm przekazywania danych kontekstowych między wątkami za pomocą *zmiennych zakresowych* (ang. *scoped values*), będący alternatywą dla klasy `ThreadLocal` w systemach współbieżnych i opisany w dokumencie JEP 506 [10].

Początkowo, ideą projektu było wprowadzenie lekkich wątków użytkownika (ang. *user-mode threads*) oraz uproszczenie programowania współbieżnego, szczególnie w aplikacjach, których dużą częścią są blokujące zadania, takich jak serwery i systemy sieciowe [11]. Wraz z upływem czasu zakres projektu został rozszerzony o mechanizmy zwiększające bezpieczeństwo aplikacji współbieżnych i ustrukturyzowanie pracy z wątkami. Project Loom wciąż ewoluuje, głównie w kontekście zmian w API poszczególnych mechanizmów, lecz większość z nich jest integralną częścią JDK, dodanych w ramach oficjalnych wydań.

3.2 Wirtualne wątki (*Virtual Threads*)

Mechanizm wirtualnych wątków, we wczesnych fazach projektu określanym mianem *fibers* [11], to chronologicznie pierwsza z trzech głównych składowych projektu Loom, która pod tą nazwą po raz pierwszy pojawiła się w JDK w wersji 19 w ramach wersji poglądowej (ang. *preview*), a została oficjalnie wydana w ramach JDK 21. Mechanizm ten jest alternatywą dla wątków platformowych, opisanych w sekcji 2.3.1 Klasyczny model współbieżności (Thread API).

W celu wyjaśnienia potrzeby wprowadzenia mechanizmu wirtualnych wątków do środowiska JVM, dokument *JEP 444: Virtual Threads* [8] powołuje się na prawo Little’a, które wiąże ze sobą latencję, współbieżność i przepustowość w kontekście skalowalności aplikacji współbieżnych:

Dla danego czasu przetwarzania żądania (tj. latencji) liczba żądań obsługiwanych jednocześnie przez aplikację (tj. współbieżność) musi rosnąć proporcjonalnie do tempa napływu żądań (tj. przepustowości).

Jak wspomniano w sekcji 2.3.1 Klasyczny model współbieżności (Thread API), wątki platformowe w środowisku JVM są zaimplementowane jako obiekty opakowujące (ang. *wrappers*) wątki systemu operacyjnego. Wątki te są zasobożerne, co uniemożliwia ich tworzenie w bardzo dużych ilościach, przez co liczba wątków często staje się czynnikiem ograniczającym skalowalność, na długo przed wyczerpaniem innych zasobów.

Próbą rozwiązania problemu zasobożerności wątków platformowych jest zastosowanie podejścia opartego na współdzieleniu wątków i programowaniu asynchronicznym (patrz sekcja 2.3.3 Reaktywny model współbieżności (Reactive Streams, Project

Reactor, RxJava)), które umożliwia obsługę dużej liczby współbieżnych operacji przy ograniczonej liczbie wątków systemowych. Rozwiązanie to eliminuje problem kosztowności wątków systemu operacyjnego, jednak odbywa się kosztem znacznego wzrostu złożoności programowania. Asynchroniczny styl wymusza dekompozycję logiki aplikacyjnej na etapy realizowane w postaci wyrażeń lambda, co uniemożliwia wykorzystanie modelu współbieżności obecnego do tej pory w środowisku JVM.

Wirtualne wątki umożliwiają zachowanie paradygmatu *thread-per-request*³ przy jednoczesnym osiągnięciu skalowalności porównywalnej z podejściem asynchronicznym.

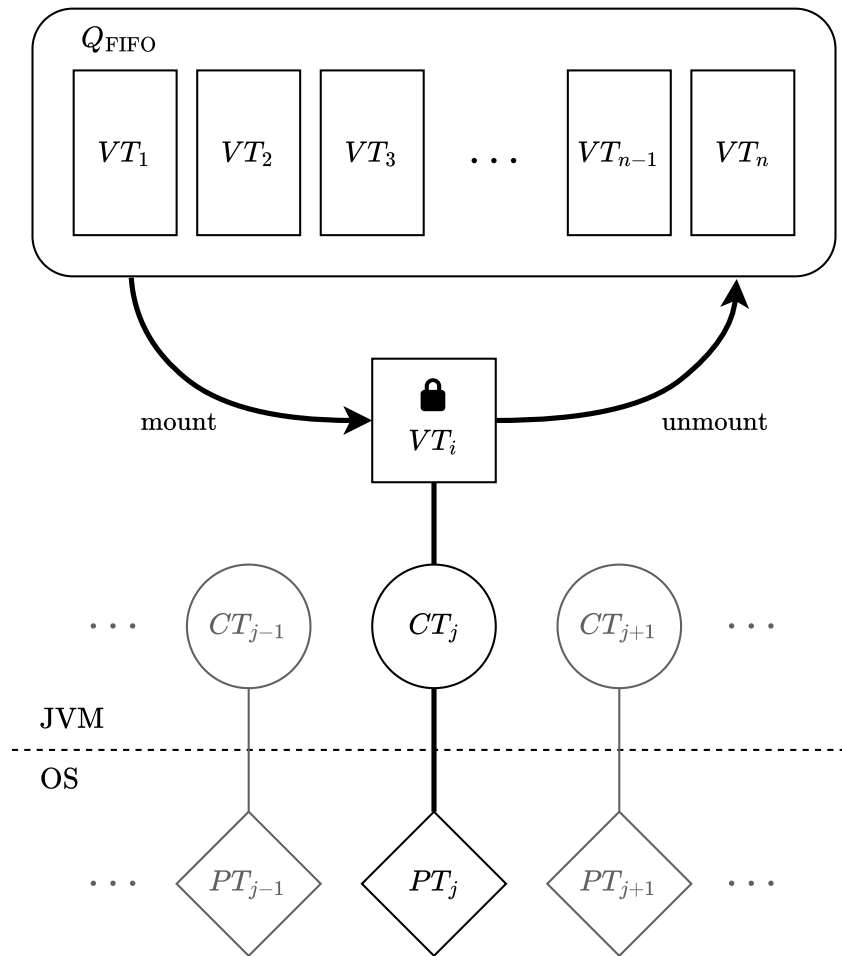
Cykl życia wirtualnych wątków

Każdorazowo, gdy tworzony jest wirtualny wątek, zostaje on dodany do *kolejki wirtualnych wątków*, będącą kolejką typu FIFO (ang. *First In, First Out*). Wirtualny wątek może wykonać przydzielone mu zadanie, gdy zostanie przypisany do *wątku nośnego* (ang. *carrier thread*). Reprezentuje on wątek platformowy (tj. wątek systemu operacyjnego) po stronie platformy JVM [3]. Każdy wątek nośny ma przypisany odpowiadający mu wątek platformowy i domyślnie ich liczba jest równa liczbie dostępnych (fizycznych) procesorów logicznych [12]. W odróżnieniu od wątków platformowych i nośnych, liczba wirtualnych wątków nie jest zależna od liczby dostępnych procesorów logicznych, a w większości przypadków ją przekracza. Wynika to ze sposobu zarządzania cyklem ich życia, który został przedstawiony na rysunku 3.1.

Korzystając z oznaczeń zawartych na rysunku 3.1, cykl życia wirtualnych wątków w środowisku JVM w kontekście jednego wątku nośnego można opisać następująco:

1. Każdy nowy wirtualny wątek VT zostaje dodany na koniec kolejki wirtualnych wątków Q_{FIFO}
2. Planista (ang. *scheduler*) wątków wirtualnych, obecny w środowisku JVM, bierze pierwszy wolny wątek wirtualny z kolejki przypisuje go (ang. *mount*) do wolnego wątku nośnego (CT_j)
3. Wątek nośny CT_j , odpowiadający wątkowi platformowemu PT_j , wykonuje kod przypisany do przydzielonego mu wątku wirtualnego VT_i
4. Gdy wątek wirtualny VT_i wykonuje operację blokującą (stan reprezentowany przez symbol kłódki ) , JVM zapisuje jego aktualny stan, dodaje go na koniec kolejki Q_{FIFO} , jednocześnie zwalniając (ang. *unmount*) wątek nośny CT_j

³Paradygmat *thread-per-request* zakłada, że każde przychodzące żądanie jest obsługiwane przez dedykowany wątek.



Rysunek 3.1: Cykl życia wirtualnych wątków w środowisku JVM
(źródło: opracowanie własne na podstawie [3])

5. JVM przypisuje kolejny wątek wirtualny (tj. pierwszy w kolejce Q_{FIFO}) do wątku nośnego CT_j

3.3 Współbieżność strukturalna (*Structured Concurrency*)

3.4 Zmienne zakresowe (*Scoped Values*)

Rozdział 4

Stan badań i analiza literatury

4.1 Dotychczasowe badania nad Project Loom

4.2 Charakterystyka i wyniki dotychczasowych badań

4.2.1 Obsługa dużej liczby równoczesnych żądań HTTP

4.2.2 Równoległe przetwarzanie złożonych obliczeń

4.3 Zdefiniowanie luki badawczej i kierunku własnych badań

Rozdział 5

Metodyka badań

5.1 Cel i zakres badań

5.2 Szczegółowy opis scenariuszy testowych

5.2.1 Scenariusz A: Integracja z bazą danych ze sterownikiem blokującym (JDBC) i reaktywnym (R2DBC)

5.2.2 Scenariusz B: Agregacja z użyciem mechanizmu kworum (QS) i propagacją kontekstu

Rozdział 6

Wyniki badań i analiza

6.1 Metody pomiaru i kryteria oceny

6.2 Wyniki pomiarów dla poszczególnych scenariuszy

Uwagi do poniższych tabel:

1. Wyniki benchmarku przeprowadzonego w trybie **Throughput** (testu przepustowości), tj. w tabelach 6.1, 6.2 i 6.3, podane są jako liczba operacji na sekundę (wykresy przedstawione na rysunkach 6.1, 6.2 i 6.3 prezentują te dane na osi pionowej w jednostce oznaczonej jako ops/s). Operacje wykonane w ramach każdego benchmarku opisane zostały w rozdziale X.
2. Wyniki w tabelach wspomnianych w punkcie 1. przedstawione są w formacie $S \pm E$, gdzie S oznacza wynik, a E – błąd pomiaru.
3. Wyniki we wszystkich tabelach zapisane zostały z kropką jako separatorem dziesiętnym, z dokładnością do części tysięcznych, tj. 0.001.

6.3 Analiza porównawcza i interpretacja wyników

Poz. współb.	JDBC + PT	R2DBC	JDBC + VT
16	415.881 ± 8.497	729.356 ± 5.935	733.642 ± 13.414
32	230.098 ± 7.511	451.219 ± 2.474	489.411 ± 2.623
64	116.672 ± 4.463	240.370 ± 4.193	250.862 ± 0.876
256	28.871 ± 1.067	67.178 ± 0.352	66.336 ± 0.378
1024	6.694 ± 0.306	17.139 ± 0.105	16.782 ± 0.111

Tabela 6.1: Wartości przepustowości dla zapytania Q1 w zależności od poziomu współbieżności i wariantu modelu współbieżności

(źródło: opracowanie własne)

Poz. współb.	JDBC + PT	R2DBC	JDBC + VT
16	142.817 ± 24.898	174.474 ± 36.850	159.069 ± 33.964
32	107.766 ± 14.097	122.683 ± 0.635	157.456 ± 29.941
64	55.572 ± 5.507	74.120 ± 10.150	70.235 ± 4.447
256	14.765 ± 1.268	19.392 ± 2.232	19.884 ± 1.825
1024	3.736 ± 0.365	4.600 ± 0.031	5.042 ± 0.379

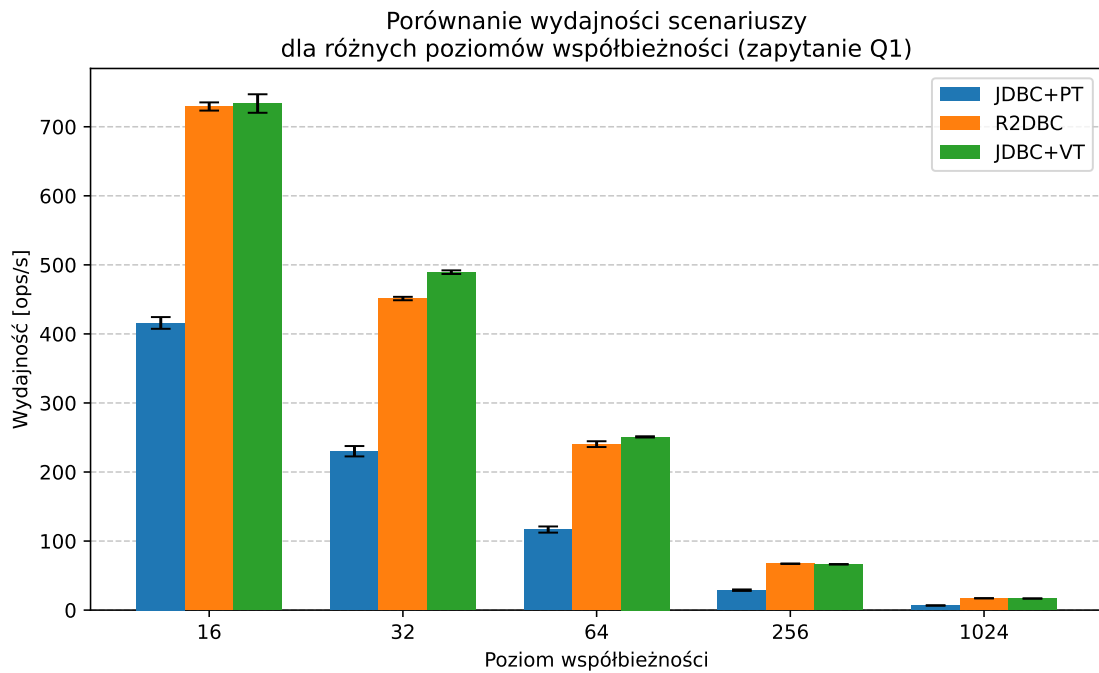
Tabela 6.2: Wartości przepustowości dla zapytania Q2 w zależności od poziomu współbieżności i wariantu modelu współbieżności

(źródło: opracowanie własne)

Poz. współb.	JDBC + PT	R2DBC	JDBC + VT
16	4.943 ± 0.020	5.003 ± 0.027	4.940 ± 0.027
32	2.667 ± 0.009	2.705 ± 0.011	2.673 ± 0.010
64	1.319 ± 0.004	1.332 ± 0.006	1.314 ± 0.008
256	0.330 ± 0.002	0.335 ± 0.001	0.330 ± 0.002
1024	0.187 ± 0.001	0.084 ± 0.001	0.192 ± 0.001

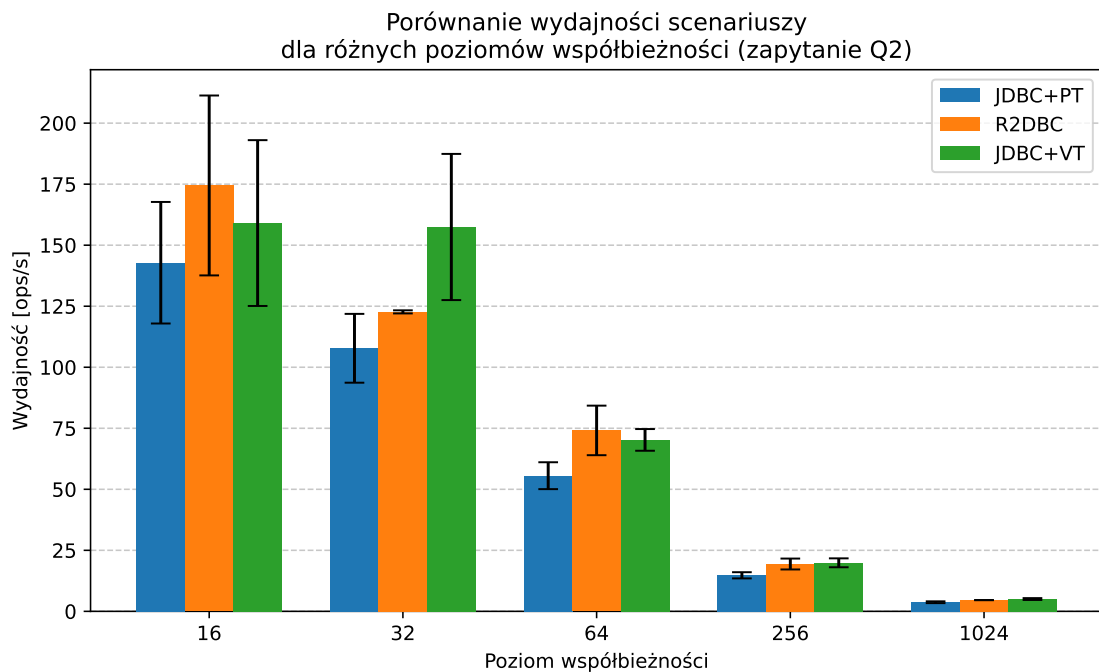
Tabela 6.3: Wartości przepustowości dla zapytania Q3 w zależności od poziomu współbieżności i wariantu modelu współbieżności

(źródło: opracowanie własne)



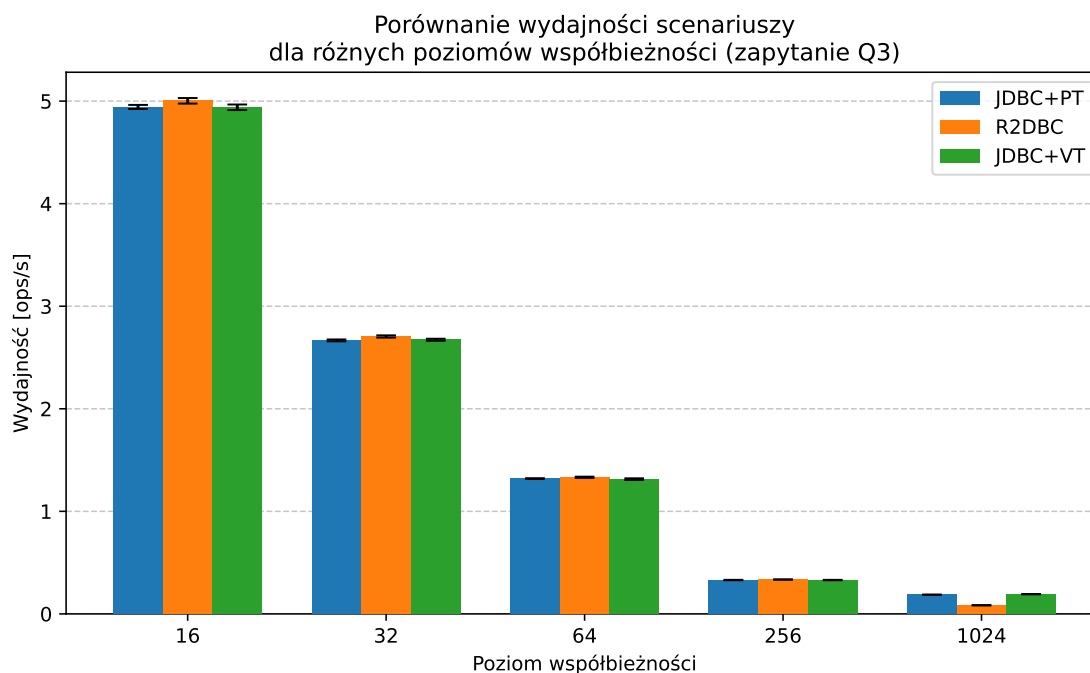
Rysunek 6.1: Wykres kolumnowy przepustowości dla zapytania Q1 w zależności od poziomu współbieżności i wariantu modelu współbieżności

(źródło: opracowanie własne)

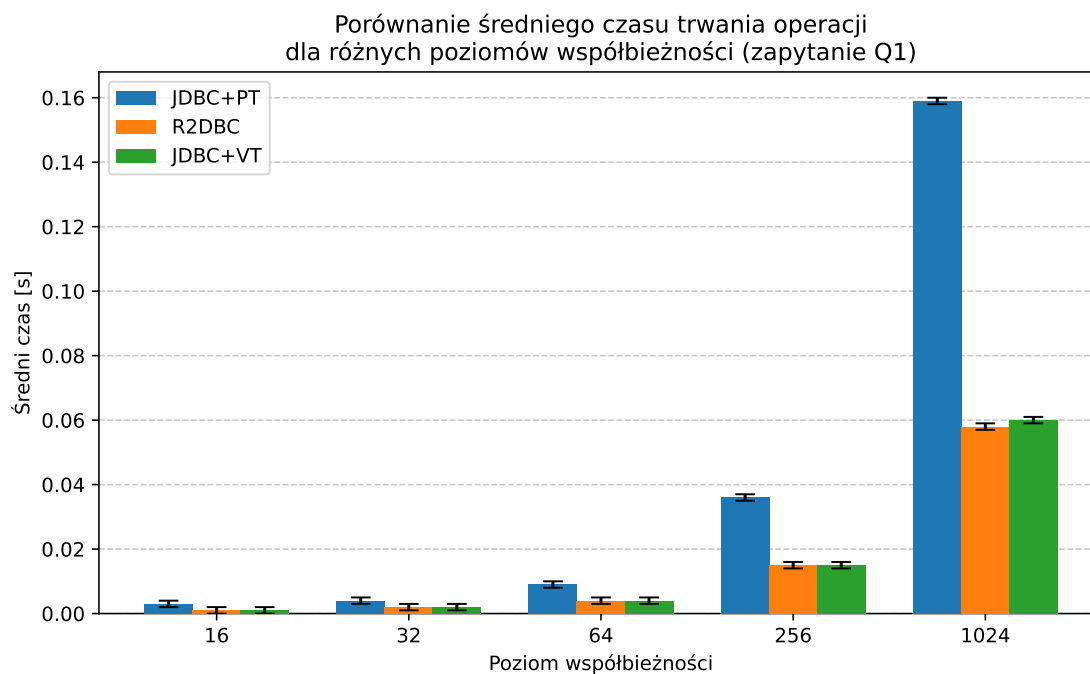


Rysunek 6.2: Wykres kolumnowy przepustowości dla zapytania Q2 w zależności od poziomu współbieżności i wariantu modelu współbieżności

(źródło: opracowanie własne)



Rysunek 6.3: Wykres kolumnowy przepustowości dla zapytania Q3 w zależności od poziomu współbieżności i wariantu modelu współbieżności



Rysunek 6.4: Wykres kolumnowy średniego czasu zapytania Q1 w zależności od poziomu współbieżności i wariantu modelu współbieżności

(źródło: opracowanie własne)

Poz. współb.	Metryka	JDBC + PT	R2DBC	JDBC + VT
16	AVG	0.003 ± 0.001	0.001 ± 0.001	0.001 ± 0.001
	P50	0.003	0.001	0.001
	P95	0.003	0.002	0.002
	P99	0.004	0.002	0.002
32	AVG	0.004 ± 0.001	0.002 ± 0.001	0.002 ± 0.001
	P50	0.004	0.002	0.002
	P95	0.005	0.003	0.002
	P99	0.006	0.004	0.003
64	AVG	0.009 ± 0.001	0.004 ± 0.001	0.004 ± 0.001
	P50	0.009	0.004	0.004
	P95	0.010	0.005	0.005
	P99	0.011	0.006	0.006
256	AVG	0.036 ± 0.001	0.015 ± 0.001	0.015 ± 0.001
	P50	0.035	0.015	0.015
	P95	0.039	0.017	0.018
	P99	0.043	0.020	0.020
1024	AVG	0.159 ± 0.001	0.058 ± 0.001	0.060 ± 0.001
	P50	0.158	0.057	0.058
	P95	0.177	0.065	0.066
	P99	0.187	0.070	0.071

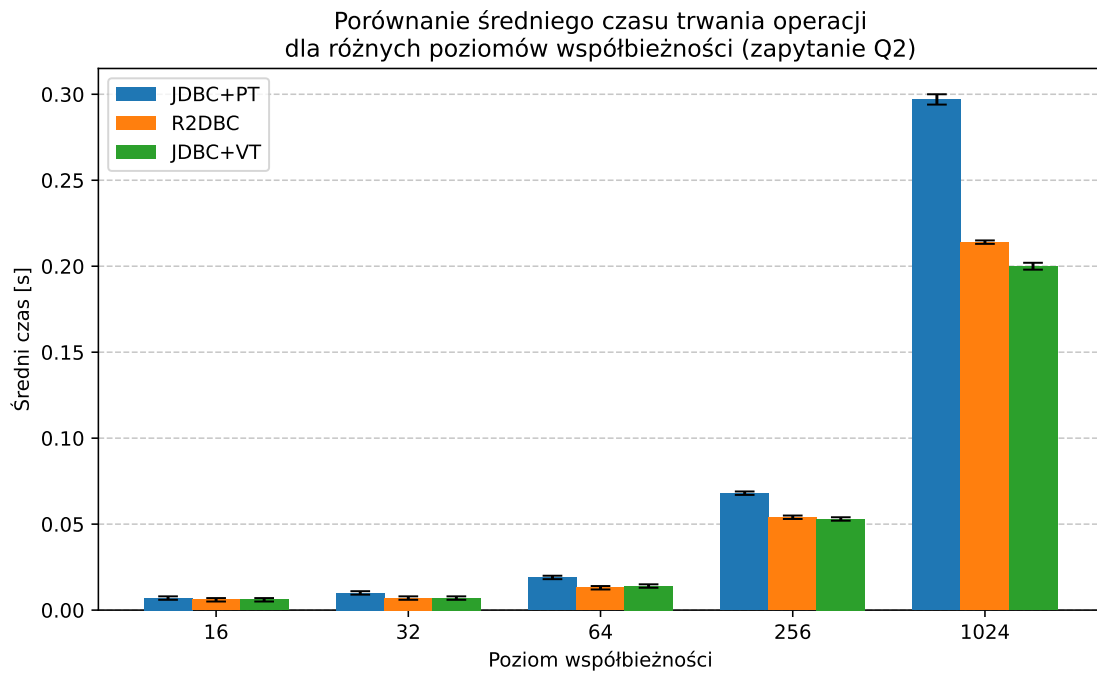
Tabela 6.4: Wartości średniego czasu i latencji P50, P95 i P99 dla zapytania Q1 w zależności od poziomu współbieżności i wariantu modelu współbieżności

(źródło: opracowanie własne)

Poz. współb.	Metryka	JDBC + PT	R2DBC	JDBC + VT
16	AVG	0.007 ± 0.001	0.006 ± 0.001	0.006 ± 0.001
	P50	0.008	0.007	0.007
	P95	0.010	0.008	0.008
	P99	0.012	0.010	0.011
32	AVG	0.010 ± 0.001	0.007 ± 0.001	0.007 ± 0.001
	P50	0.010	0.008	0.008
	P95	0.011	0.009	0.009
	P99	0.014	0.011	0.011
64	AVG	0.019 ± 0.001	0.013 ± 0.001	0.014 ± 0.001
	P50	0.019	0.014	0.014
	P95	0.021	0.016	0.016
	P99	0.025	0.020	0.022
256	AVG	0.068 ± 0.001	0.054 ± 0.001	0.053 ± 0.001
	P50	0.071	0.054	0.052
	P95	0.077	0.059	0.056
	P99	0.096	0.081	0.079
1024	AVG	0.297 ± 0.003	0.214 ± 0.001	0.200 ± 0.002
	P50	0.299	0.212	0.202
	P95	0.329	0.236	0.223
	P99	0.364	0.264	0.257

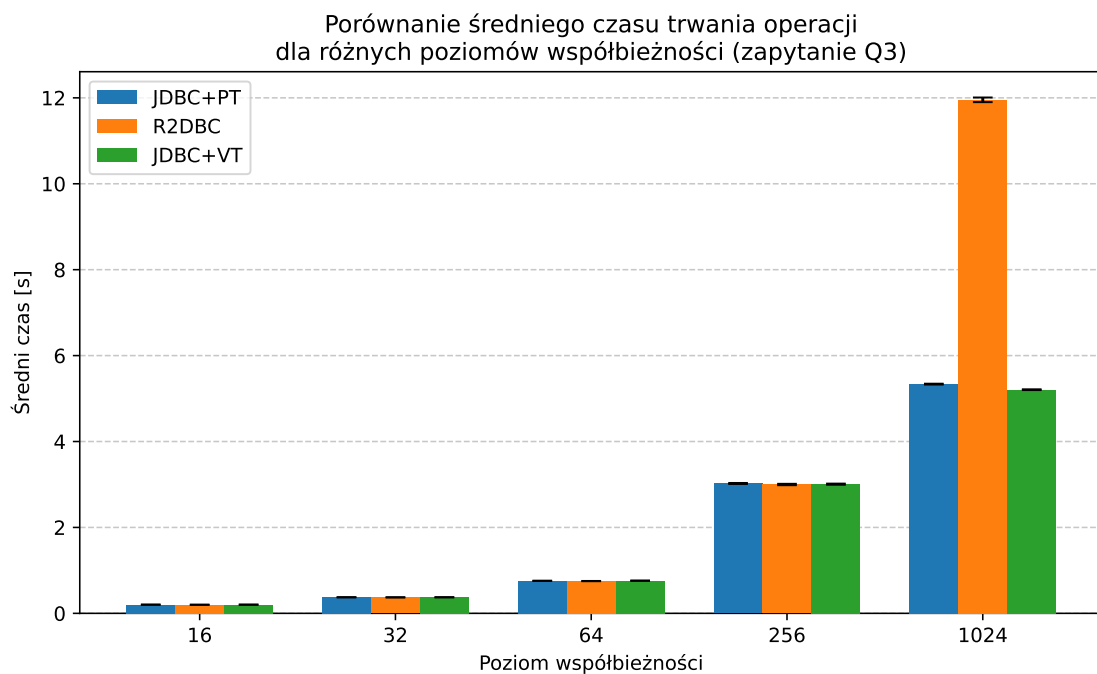
Tabela 6.5: Wartości średniego czasu i latencji P50, P95 i P99 dla zapytania Q2 w zależności od poziomu współbieżności i wariantu modelu współbieżności

(źródło: opracowanie własne)



Rysunek 6.5: Wykres kolumnowy średniego czasu zapytania Q2 w zależności od poziomu współbieżności i wariantu modelu współbieżności

(źródło: opracowanie własne)



Rysunek 6.6: Wykres kolumnowy średniego czasu zapytania Q3 w zależności od poziomu współbieżności i wariantu modelu współbieżności

(źródło: opracowanie własne)

Poz. współb.	Metryka	JDBC + PT	R2DBC	JDBC + VT
16	AVG	0.202 ± 0.001	0.200 ± 0.001	0.202 ± 0.001
	P50	0.200	0.199	0.200
	P95	0.217	0.216	0.218
	P99	0.227	0.226	0.228
32	AVG	0.374 ± 0.001	0.372 ± 0.001	0.374 ± 0.001
	P50	0.371	0.369	0.372
	P95	0.400	0.394	0.394
	P99	0.412	0.412	0.406
64	AVG	0.757 ± 0.003	0.751 ± 0.003	0.760 ± 0.003
	P50	0.753	0.748	0.755
	P95	0.784	0.778	0.793
	P99	0.820	0.820	0.830
256	AVG	3.023 ± 0.012	2.999 ± 0.016	3.007 ± 0.013
	P50	3.012	2.991	3.001
	P95	3.091	3.078	3.095
	P99	3.176	3.311	3.148
1024	AVG	5.336 ± 0.010	11.954 ± 0.054	5.206 ± 0.009
	P50	5.335	11.929	5.201
	P95	5.377	12.148	5.234
	P99	5.385	12.231	5.260

Tabela 6.6: Wartości średniego czasu i latencji P50, P95 i P99 dla zapytania Q3 w zależności od poziomu współbieżności i wariantu modelu współbieżności

(źródło: opracowanie własne)

Rozdział 7

Wnioski i rekomendacje

7.1 Odpowiedzi na pytania badawcze

7.2 Rekomendacje i wzorce zastosowań mechanizmów Project Loom

7.3 Ograniczenia badań i propozycje dalszych badań

Rozdział 8

Podsumowanie

Stworzenie aplikacji *Grocery Scanner* wymagało ogromnego nakładu czasowego. Proces jej powstawania był pełen różnych koncepcji i pomysłów na rozbudowę aplikacji. Wiele z nich często podlegało zmianom, lecz efekt końcowy pozwala stwierdzić, że aplikacja spełnia wyrażone we wstępie założenia. Możliwe jest jednak poszerzanie jej o dodatkowe funkcjonalności. Przykładowymi planami na rozwój aplikacji są:

- *możliwość logowania i rejestracji za pomocą konta w takich serwisach, jak Google i Facebook*, co usprawni zaimplementowany już proces, a w konsekwencji pozwoli na powiększenie liczby użytkowników korzystających z aplikacji,
- *system zgłaszania profili użytkowników*, który pozwoli na ograniczenie szkodliwej działalności w aplikacji poprzez ograniczanie lub blokowanie dostępu,
- *rozszerzenie listy preferencji żywieniowych*, która ułatwi korzystanie z aplikacji większej liczbie użytkowników,
- *pobieranie informacji o produktach z innych źródeł*, co pozwoli na wzrost jakości danych, a także w dużym stopniu ograniczy przypadki, w których aplikacja zwraca informację o braku danych skanowanego produktu.

Warto zaznaczyć, że funkcjonalności zawarte w powyższej liście nie zostały zaimplementowane z uwagi na fakt, że wykraczały poza podstawowe założenia co do funkcjonalności aplikacji opisanych we wstępie pracy.

Proces tworzenia niniejszej pracy przede wszystkim pozwolił na pogłębienie wiedzy. W tej kwestii szczególną uwagę należy zwrócić na aspekt „techniczny”. Tworzenie aplikacji *Grocery Scanner* pozwoliło na poznanie nieużywanych wcześniej przez autora pracy bibliotek i narzędzi, jak również umożliwiło poszerzenie wiedzy na temat poznanych wcześniej rozwiązań. Sam proces tworzenia aplikacji na urządzenia mobilne okazał się być niezwykle pouczający, zarówno w kontekście poszerzania wiedzy technicznej, jak i rozwoju kariery zawodowej.

Bibliografia

- [1] OpenAI. *ChatGPT Documentation*. <https://platform.openai.com/docs>. Dostęp: 24 września 2025.
- [2] Adam Kowalski i Anna Nowak. „Graph Query Languages for JVM”. W: *Proceedings of the International Conference on Software Engineering*. 2021, s. 101–110.
- [3] Ron Veen i David Vlijmincx. *Virtual Threads, Structured Concurrency, and Scoped Values: Explore Java’s New Threading Model*. Apress Berkeley, CA, 2024.
- [4] Andrew S. Tanenbaum i Herbert Bos. *Modern Operating Systems*. Wydanie V. Pearson Education, 2023, s. 85–178.
- [5] Mordechai Ben-Ari. *Principles of Concurrent Programming*. Prentice-Hall International, 1982.
- [6] Maurice Herlihy i Nir Shavit. *The Art of Multiprocessor Programming*. Elsevier, 2012, s. 6–10, 21–44.
- [7] Brian Goetz. *Java Concurrency in Practice*. Pearson Education, 2006.
- [8] OpenJDK. *JEP 444: Virtual Threads*. <https://openjdk.org/jeps/444>. Dostęp: 1.11.2025.
- [9] OpenJDK. *JEP 525: Structured Concurrency (Sixth Preview)*. <https://openjdk.org/jeps/525>. Dostęp: 1.11.2025.
- [10] OpenJDK. *JEP 506: Scoped Values*. <https://openjdk.org/jeps/506>. Dostęp: 1.11.2025.
- [11] Ron Pressler (OpenJDK). *Project Loom: Fibers and Continuations for the Java Virtual Machine*. <https://cr.openjdk.org/~rpressler/loom/Loom-Proposal.html>. Dostęp: 12.12.2025.
- [12] Alan Bateman (OpenJDK). *Project Loom: The challenges of introducing Virtual Threads to the Java Platform*. <https://cr.openjdk.org/~alanb/jvmls2023/loom-jvmls2023.pdf>. Dostęp: 11.03.2026.

Spis rysunków

2.1	Rozróżnienie procesów i wątków w systemie operacyjnym	10
3.1	Cykl życia wirtualnych wątków w środowisku JVM	16
6.1	Wykres kolumnowy przepustowości dla zapytania Q1 w zależności od poziomu współbieżności i wariantu modelu współbieżności	23
6.2	Wykres kolumnowy przepustowości dla zapytania Q2 w zależności od poziomu współbieżności i wariantu modelu współbieżności	23
6.3	Wykres kolumnowy przepustowości dla zapytania Q3 w zależności od poziomu współbieżności i wariantu modelu współbieżności	24
6.4	Wykres kolumnowy średniego czasu zapytania Q1 w zależności od poziomu współbieżności i wariantu modelu współbieżności	24
6.5	Wykres kolumnowy średniego czasu zapytania Q2 w zależności od poziomu współbieżności i wariantu modelu współbieżności	27
6.6	Wykres kolumnowy średniego czasu zapytania Q3 w zależności od poziomu współbieżności i wariantu modelu współbieżności	27

Spis tabel

6.1	Wartości przepustowości dla zapytania Q1 w zależności od poziomu współbieżności i wariantu modelu współbieżności	22
6.2	Wartości przepustowości dla zapytania Q2 w zależności od poziomu współbieżności i wariantu modelu współbieżności	22
6.3	Wartości przepustowości dla zapytania Q3 w zależności od poziomu współbieżności i wariantu modelu współbieżności	22
6.4	Wartości średniego czasu i latencji P50, P95 i P99 dla zapytania Q1 w zależności od poziomu współbieżności i wariantu modelu współbieżności	25
6.5	Wartości średniego czasu i latencji P50, P95 i P99 dla zapytania Q2 w zależności od poziomu współbieżności i wariantu modelu współbieżności	26
6.6	Wartości średniego czasu i latencji P50, P95 i P99 dla zapytania Q3 w zależności od poziomu współbieżności i wariantu modelu współbieżności	28

Spis kodów źródłowych

1	Uruchamianie wirtualnego wątku	7
---	--	---